

Req 3 & 4 Weather system

Design A:

Implement a dedicated WeatherSystem that manages global weather states and delegates weather-specific behaviour to classes implementing the WeatherEffect interface. Each weather type (SunnyEffect, HeatwaveEffect, StormEffect, TornadoEffect) defines its own logic for how the environment and actors are affected. The system keeps track of duration, transitions, and announcements while remaining independent of the game engine's internal logic.

Pros	Cons
Follows Single Responsibility and Open–Closed principles – system handles orchestration; effects handle behaviour.	Slightly higher complexity due to multiple small classes.
Easy to extend: new weather types can be added without modifying existing code.	Requires registration of each effect in WeatherSystem.
Testable: fake effects can be injected to verify sequencing (onStart, applyPerTurn, onEnd).	Must ensure all weather effects are registered at runtime.
Engine-agnostic: depends only on interfaces and simple types (GameMap, Actor, Display).	—

Explanation:

This modular design keeps the system flexible and easy to maintain. It avoids the “god class” issue by delegating behaviour to separate effect classes, making the code cleaner and more readable.

The WeatherSystem manages turn-based updates and transitions, while each effect focuses solely on its own gameplay logic.

Design B:

Define a small contract WeatherEffect with lifecycle hooks
onStart(GameMap), applyPerTurn(GameMap), onEnd(GameMap), name().

Each weather is a separate class that implements this contract:

SunnyEffect, HeatwaveEffect, StormEffect, TornadoEffect.

WeatherSystem acts as the orchestrator: it stores a registry

Map<WeatherType, WeatherEffect>, tracks current and turnsLeft, and on each turn calls the active effect's hook(s), announces via Display, decrements duration, and performs transition when needed.

Pros | Cons of this implemented design

Pros	Cons
Clear separation of concerns: WeatherSystem = scheduling/announcing; each effect class = its own behaviour.	More small classes to maintain; requires registering effects at startup.
Open to extension: adding a new weather is additive (create class + register), no edits to WeatherSystem.	Need to keep a consistent naming/registration convention.
Testability: effects can be unit-tested in isolation; spies/fakes validate onStart → applyPerTurn → onEnd sequencing.	—
Minimal engine coupling: effects depend only on stable engine types (GameMap, Actor, Location, Exit, Display).	—

What each implemented effect does (matches code):

- **SunnyEffect:** Baseline; no side effects (used for pacing).
- **HeatwaveEffect:** Grants HeatEmpowered to actors and increases WARMTH (capped).
- **StormEffect:** Performs limited random lightning strikes each turn. Hitting an actor: damage + MovementLocked (no movement for N rounds). Hitting empty ground: replace with DamagedGround (impassable).
- **TornadoEffect:** Samples some actors and probabilistically displaces them to accessible neighbouring tiles; successful displacement also applies damage. Logs are rate-limited to avoid spam.

Implementation notes (tie to codebase):

- `WeatherSystem.tick(map)` handles first-turn `onStart`, per-turn `applyPerTurn`, `countdown`, `onEnd`, then selects next weather and announces.
- Effects print concise messages through `Display` with limits to keep the HUD readable.
- Status effects are modelled via engine `Status`: `HeatEmpowered` (burn-on-hit) and `MovementLocked` (blocks movement-type actions).

Design C:

Model weather side-effects as engine `Status` implementations and attach them to actors when the weather applies. I implemented two statuses:

- **HeatEmpowered** — temporary; while active, successful attacks gain a burn-like on-hit effect. Integration point: `AttackAction` checks for this status and augments the attack accordingly.
- **MovementLocked** — temporary; while active, the actor is blocked from movement-type actions. Enforcement is engine-agnostic via `MovementActions.isMovement(Action)` (class-name allow-list, no instanceof).

Lifecycle & control:

Statuses store a duration and implement `tickStatus()/isStatusActive()`. Weather effects attach/refresh statuses during `applyPerTurn()` (or `onStart()` when appropriate); expiry is handled uniformly by the engine's status ticking.

Pros | Cons of this implemented design

Pros	Cons
Clean separation of cross-cutting behaviour from actions/actors; keeps <code>AttackAction</code> lean.	More small classes than using ad-hoc flags.
Open to extension: new effects are additive (new <code>Status</code> class), no edits to <code>Actor</code> .	Callers must attach/refresh statuses at the right hook (<code>onStart/applyPerTurn</code>).

Testable: can unit-test each status and verify AttackAction / movement filtering with/without the status.	—
---	---

Implementation notes (tie to code):

- HeatEmpowered.java & MovementLocked.java implement Status with duration, tickStatus, isActive.
- AttackAction applies burn-on-hit when HeatEmpowered is present.
- Movement blocking uses MovementActions.isMovement(Action) so we don't depend on engine internals.

Design D:

Represent lightning-damaged tiles as a dedicated terrain class DamagedGround extends Ground. It is permanently impassable and created by StormEffect when a lightning strike hits an empty tile. A safe helper replaceIfFree(Location) ensures we never “trap” an actor on a newly impassable cell.

Behaviour (matches code):

- canActorEnter(Actor) returns false.
- tick(Location) is a no-op (terrain is stable once created).
- replaceIfFree(Location) only substitutes ground when no actor occupies the cell.
- StormEffect.applyPerTurn(map) randomly selects tiles; if unoccupied, replaces with DamagedGround; otherwise, if an actor is present, it deals damage and applies MovementLocked.

Pros | Cons of this implemented design

Pros	Cons
Clear semantic type for post-storm terrain; easy to recognise in code and on the map.	Adds one more terrain class to maintain.

Prevents soft-locks by guarding replacement with <code>replaceIfFree</code> .	Requires callers (Storm) to use the safe replacement API correctly.
Keeps <code>StormEffect</code> simple: just call <code>setGround(new DamagedGround())</code> or <code>replaceIfFree</code> .	Terrain has no decay — stays forever unless extra logic is added.

Implementation notes (tie to code):

- Class: `game.positions.DamagedGround` with symbol '*' and name "Damaged Ground".
- Created exclusively by `StormEffect` on empty-strike tiles; actor-strike path applies damage + `MovementLocked` instead.
- No side-effects during ticks; behaviour is fully determined at creation time.

Design E:

Integrate a single `WeatherSystem` instance into the map/earth layer so that weather advances once per game turn and announcements appear on the HUD.

- Ownership: Earth (or the map owner) creates and holds the `WeatherSystem`.
- Wiring: On initialisation, register all effects `register(SUNNY, new SunnyEffect().bindDisplay(display))`, etc., and set the announcer via `setAnnouncer(Display)`.
- Ticking: Each map turn, Earth calls `weatherSystem.tick(GameMap)` to run the lifecycle (`onStart` / `applyPerTurn` / `onEnd`) and handle transitions/duration.
- Action filtering: Movement blocking is engine-agnostic via `MovementActions.isMovement(Action)`, so we do not depend on engine concrete classes or `instanceof`.

Pros | Cons of this implemented design

Pros	Cons
------	------

Clear turn-based entry point: one call from Earth per tick; easy to reason about sequencing.	Requires the map owner to remember calling tick(map) every turn.
Centralised announcements through Display; consistent HUD output.	Display dependency must be bound early (during setup).
Engine-agnostic integration: only small stable engine types are referenced (GameMap, Actor, Location, Exit, Display).	Name-list in MovementActions must be maintained when introducing new movement actions.
Supports multiple maps by instantiating one WeatherSystem per GameMap if needed.	—

Implementation notes (tie to code):

- WeatherSystem stores current, turnsLeft, and an announcer (Display); it announces on first turn of a weather and on change.
- Duration uses a small fixed value (3) to keep demos predictable; transition avoids repeating the previous weather.
- Effects bind to Display via bindDisplay(display) to print concise, rate-limited messages.

applyPerTurn, onEnd, and name. Internally, WeatherSystem aggregates a registry Map<WeatherType, WeatherEffect>, tracks the current weather and remaining duration, and depends on GameMap (for ticking) and Display (for announcements). This separation keeps orchestration small and stable while each effect evolves independently (SRP, OCP, DRY).

The concrete effect classes encapsulate only their own gameplay impact, exactly as shown in the diagram. HeatwaveEffect grants a temporary HeatEmpowered status to actors and increases warmth up to a cap; StormEffect performs bounded random lightning strikes that either damage actors and apply MovementLocked or convert empty tiles to DamagedGround; TornadoEffect probabilistically displaces actors to accessible neighbouring tiles by using Location/Exit and applies damage; SunnyEffect is a neutral baseline for pacing. Terrain change is modelled structurally with DamagedGround —|> Ground, and the base terrain Earth —|> Ground appears in the positions package to reflect the actual map ground types.

Weather-driven actor states are modelled as engine Status implementations: HeatEmpowered and MovementLocked both realise Status and follow the engine lifecycle (tickStatus, isStatusActive). Because statuses are additive and self-contained, new effects can be introduced without modifying actors or engine code (OCP). Integration points are minimal and explicit: AttackAction —|> Action and checks HeatEmpowered to apply burn-on-hit, while movement is blocked when MovementLocked is active.

To remain engine-agnostic, movement detection uses the small utility MovementActions, which depends only on Action and classifies movement-type actions by class name rather than instanceof. Finally, the world layer (e.g., Earth) owns a WeatherSystem instance and calls weatherSystem.tick(map) once per turn, which cleanly integrates weather into the game loop without touching player code. Overall, the design in the UML maps 1-for-1 to the implemented classes and relations: aggregation from WeatherSystem to WeatherEffect, realisation for effects and statuses, generalisation for grounds and actions, and a minimal set of dependencies for map access, announcements, and safe displacement.